

## Задание 1.

Management (“Управление”) в MySQL Workbench предоставляет пользователю доступ к различным инструментам, которые позволяют управлять экземпляром БД.

- Server Status - отображает информацию о текущем состоянии сервера БД, включая количество соединений, использование ресурсов процессора, объем свободного места на диске и т.д.
- Client Connections - позволяет просмотреть информацию о текущих соединениях с сервером БД, включая IP-адрес клиента, количество запросов и т.д.
- Users and Privileges - позволяет управлять пользователями и привилегиями, задаваемыми им на уровне экземпляра БД.
- Status and System Variables - позволяет просмотреть текущие настройки системы, включая параметры конфигурации, установленные плагины, названия таблиц и т.д.
- Data Export - позволяет экспортировать данные из БД в различные форматы, например, в формат CSV или XML.
- Data Import/Restore - позволяет импортировать данные в БД из различных источников, а также восстановить данные из резервной копии.

Пункт меню Instance (“Экземпляр БД”) позволяет управлять экземпляром БД.

- Startup / Shutdown - позволяет запустить или остановить экземпляр БД.
- Server Logs - позволяет просматривать логи, создаваемые экземпляром БД, включая ошибки и предупреждения.
- Options File - позволяет изменять параметры конфигурации, управляющие поведением экземпляра БД.

Пункт меню Performance (“Производительность”) предоставляет инструменты для анализа производительности экземпляра БД.

- Dashboard - позволяет отслеживать использование ресурсов процессора, оперативной памяти и дискового пространства.
- Performance Reports - позволяет генерировать отчеты о производительности, включая информацию о запросах, времени ответа и т.д.
- Performance Schema Setup - позволяет настроить Performance Schema, инструмент, который собирает информацию о производительности запросов и других операций в БД.

## Задание 2.

```
CREATE DATABASE `simplifiedb` /*!40100 DEFAULT CHARACTER SET utf8mb3 */ /*!80016 DEFAULT  
ENCRYPTION='N' */;
```

Задание 3.

```
CREATE TABLE `users` (  
  `id` int NOT NULL AUTO_INCREMENT,  
  `name` varchar(45) NOT NULL,  
  `email` varchar(45) NOT NULL,  
  PRIMARY KEY (`id`),  
  UNIQUE KEY `email_UNIQUE` (`email`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb3;
```

## Задание 4.

```
INSERT INTO `simplifiedb`.`users` (`name`, `email`) VALUES ('Миша', 'exemple01@mail.ru');  
INSERT INTO `simplifiedb`.`users` (`name`, `email`) VALUES ('Женя', 'exemple02@mail.ru');  
INSERT INTO `simplifiedb`.`users` (`name`, `email`) VALUES ('Алина', 'exemple03@mail.ru');  
  
UPDATE `simplifiedb`.`users` SET `name` = 'Михаил' WHERE (`id` = '1');  
UPDATE `simplifiedb`.`users` SET `name` = 'Евгений', `email` = 'exemple04@mail.ru' WHERE (`id` = '2');
```

### Задание 5.

```
ALTER TABLE simplifiedb . users
ADD COLUMN `gender` ENUM('M', 'F') NULL AFTER `email`,
ADD COLUMN `bday` DATE NOT NULL AFTER `gender`,
ADD COLUMN `postal_code` VARCHAR(10) NULL AFTER `bday`,
ADD COLUMN `rating` FLOAT NULL AFTER `postal_code`,
ADD COLUMN `created` TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP() AFTER `rating`,
CHANGE COLUMN `name` `name` VARCHAR(50) NOT NULL ;
```

```
ALTER TABLE `simplified`.`users`  
CHANGE COLUMN `bday` `bday` DATE NOT NULL ,  
CHANGE COLUMN `rating` `rating` FLOAT NOT NULL DEFAULT 0 ;
```

Поле `created` с типом данных `TIMESTAMP` и значением по умолчанию `CURRENT_TIMESTAMP()` означает, что при вставке новой записи в таблицу, если не указано явно значение для поля `created`, то в это поле будет автоматически записана текущая дата и время в формате `YYYY-MM-DD HH:MM:SS`.

Функция `CURRENT_TIMESTAMP()` возвращает текущую дату и время на сервере базы данных в формате `YYYY-MM-DD HH:MM:SS`. Таким образом, значение по умолчанию для поля `created` будет отличаться для разных вставок, так как оно будет соответствовать моменту создания записи в базе данных.

Этот подход позволяет автоматически отслеживать момент создания записи без необходимости явного указания этой информации при каждой вставке.

### Задание 6.

[illegible]

## Задание 7.

```
1  /*
2  -- Query: SELECT * FROM simplifiedb.users
3  LIMIT 0, 1000
4
5  -- Date: 2024-06-03 04:23
6  */
7  • INSERT INTO `` ('id','name','email','gender','bday','postal_code','rating','created') VALUES (1,'Михаил','exemple01@mail.ru',NULL,'2000-03-03',NULL,0,'2024-06-03 04:07:09');
8  • INSERT INTO `` ('id','name','email','gender','bday','postal_code','rating','created') VALUES (2,'Евгений','exemple04@mail.ru',NULL,'2002-11-20',NULL,0,'2024-06-03 04:07:09');
9  • INSERT INTO `` ('id','name','email','gender','bday','postal_code','rating','created') VALUES (3,'Алина','exemple03@mail.ru',NULL,'2002-09-27',NULL,0,'2024-06-03 04:07:09');
10 • INSERT INTO `` ('id','name','email','gender','bday','postal_code','rating','created') VALUES (4,'Ekaterina','ekaterina.petrova@outlook.com','F','2000-02-11','145789',1.123,'2024-06-03 04:13:38');
11 • INSERT INTO `` ('id','name','email','gender','bday','postal_code','rating','created') VALUES (5,'Paul','paul@superpochta.ru','M','1998-08-12','123789',1,'2024-06-03 04:13:38');
12
```

## Задание 8.

```
1  CREATE TABLE `simplifiedb`.`resume` (
2      `resumeid` INT NOT NULL AUTO_INCREMENT,
3      `userid` INT NOT NULL,
4      `title` VARCHAR(100) NOT NULL,
5      `skills` TEXT NULL,
6      `created` TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP(),
7      PRIMARY KEY (`resumeid`),
8      INDEX `userid_idx` (`userid` ASC) VISIBLE,
9      CONSTRAINT `userid`
10     FOREIGN KEY (`userid`)
11     REFERENCES `simplifiedb`.`users` (`id`)
12     ON DELETE CASCADE
13     ON UPDATE CASCADE);
14
```

Параметр Cascade при операциях On Update и On Delete определяет, что произойдет с зависимыми записями в других таблицах, которые ссылаются на удаляемую или обновляемую запись в данной таблице.

Если параметр Cascade установлен для On Update, то при изменении значения первичного ключа в записи таблицы, все внешние ключи в других таблицах, которые ссылаются на эту запись, будут автоматически обновлены, чтобы отразить новое значение первичного ключа.

Если параметр Cascade установлен для On Delete, то при удалении записи из таблицы, все зависимые записи в других таблицах, которые ссылаются на эту запись, будут также удалены.

Таким образом, установка параметра Cascade позволяет автоматически обновлять или удалять зависимые записи в других таблицах при обновлении или удалении записей в данной таблице, что может быть очень удобным в тех случаях, когда требуется поддерживать целостность данных в базе данных. Однако, следует быть осторожным при использовании параметра Cascade, чтобы избежать неожиданных изменений в данных и потери целостности.

## Задание 9.

```
1  /*
2  -- Query: SELECT * FROM simpledb.resume
3  LIMIT 0, 1000
4
5  -- Date: 2024-06-03 04:50
6  */
7  * INSERT INTO `` ('resumeid','userid','title','skills','created') VALUES (3,1,'Почти Developer','Python, C#, SQLite','2024-06-03 04:50:02');
8  * INSERT INTO `` ('resumeid','userid','title','skills','created') VALUES (4,2,'Человек-мультитал','Оператор, Режиссер, Коллектор, Гроссмейстер, Директор, Плотник, Асфальтоукладчик, Пивной сомелье, Раздатчик соусов, Директор', '2024-06-03 04:50:02');
9  * INSERT INTO `` ('resumeid','userid','title','skills','created') VALUES (5,2,'Подбиратель подарков','Колористика, Дизайнерский крой, Широкие плечи, Чувство стиля, Загребание руки, Просто классный мужик','2024-06-03 04:50:02');
10 * INSERT INTO `` ('resumeid','userid','title','skills','created') VALUES (6,3,'Психолог-педагог','Умение разговаривать, Знаю что-то про детей','2024-06-03 04:50:02');
11
```

Попытка добавить запись в таблицу resume с несуществующим userid не пройдет, так как userid является внешним ключом, связанным с полем id в таблице users. Если в таблице users нет записи с соответствующим id, то добавление записи в таблицу resume с таким userid вызовет ошибку, связанную с нарушением целостности данных.

Если userid в таблице resume является внешним ключом, связанным с primary key атрибутом id в таблице users, то количество резюме, которые может иметь пользователь, зависит от ограничений, которые могут быть установлены для данной связи.

В данном случае, если предполагается, что каждый пользователь может иметь множество резюме, то ограничений на количество резюме, которые может иметь пользователь, не будет. То есть, в теории, пользователь может иметь любое количество резюме, которое может поместиться в базу данных.

## Задание 10.

```
1  DELETE FROM `simpledb`.`users` WHERE (`id` = '3');
2
1  UPDATE `simpledb`.`users` SET `id` = '1' WHERE (`id` = '7');
```

Параметр Cascade, который был установлен для внешнего ключа в таблице resume, указывает, что при удалении или изменении связанной записи в таблице users, все связанные записи в таблице resume также будут удалены или обновлены соответствующим образом.

Если в таблице users будет удалена запись с id=3, то все связанные записи в таблице resume, где userid=3, будут также удалены. Это произойдет благодаря параметру ON DELETE CASCADE, который указывает на автоматическое удаление связанных записей в таблице resume при удалении записи в таблице users.

Также, если в таблице users будет изменен id пользователя, например с id = 7 на id = 1, то все связанные записи в таблице resume, где userid = 7, будут изменены на userid = 1. Это произойдет благодаря параметру ON UPDATE CASCADE, который указывает на автоматическое изменение связанных записей в таблице resume при изменении записи в таблице users.

